Software Test Plan (STP)

Railway Reservation System (Booking & Cancellation)

Project: Railway Reservation System (Booking & Cancellation)

Version: 1.0

Authors: QA Team — Team 15 (Pranav B N, Samaresh, Amruta Karoshi, Manapriya Kudurebyalya)

Date: 16-09-2026

Status: Draft

Reference SRS: Railway Reservation System SRS v1.0, dated 13-09-2026

---

1. Introduction

Purpose: This document defines the test plan for the Railway Reservation System v1.0, a C/C++ console application built as a systems-programming exercise. It outlines the objectives, scope, strategy, resources, schedule, and responsibilities for testing the booking, cancellation, search, admin, and data-persistence functionality described in the SRS.

Scope: Testing covers passenger-facing features (registration, login, train search, seat availability, ticket booking, PNR generation, simulated payment, cancellation, refund calculation, waitlist/RAC promotion, PNR status lookup) and the admin/clerk console (train, fare and quota management, reporting, backup/restore). Multi-user concurrency, a real payment gateway, mobile apps, and integration with actual IRCTC infrastructure are out of scope, matching the SRS constraints.

References: Railway Reservation System SRS v1.0 (13-09-2026), UML use-case diagrams (Sections 7.1, 7.2 of the SRS), Requirements Traceability Matrix (Section 8 of the SRS).

Definitions: PNR (Passenger Name Record), RAC (Reservation Against Cancellation), WL (Waitlist), CLI (Command Line Interface), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), FR/NFR (Functional/Non-Functional Requirement).

2. Test Items

- Authentication module (passenger registration, passenger login, admin login)
- Train Search & Seat Availability module
- Ticket Booking module (passenger details, PNR generation, seat decrement, RAC/Waitlist, simulated payment, receipt)
- Ticket Cancellation & Refund module (PNR-based cancellation, refund slab calculation, waitlist promotion)
- PNR Status & Ticket Lookup module
- Admin console (train/fare/quota management, deactivation, reporting)
- Data Persistence & Recovery module (file I/O, backup/restore)

3. Features to be Tested

Features mapped to SRS requirement IDs:

Req ID	Feature
RRS-F-001–003	Passenger registration, passenger login, admin login
RRS-F-010–012	Train search, seat availability display, RAC/Waitlist hint on full class
RRS-F-020–025	Passenger detail capture, PNR generation, seat decrement, RAC/Waitlist queuing, simulated payment, ticket receipt

Req ID	Feature
RRS-F-030–034	Cancellation by PNR, refund slab calculation, waitlist auto-promotion, cancelled-status retention, PNR status lookup
RRS-F-040–043	Add/update/deactivate train, booking & revenue reporting
RRS-F-050–051	Persist state to disk, backup/restore
RRS-NF-001–005	Search response time (≤1s), no crash on invalid input, file consistency after abrupt termination, plain-language CLI, modular codebase
RRS-SR-001–005	Hashed password storage, masked input, admin menu access control, admin action logging, login lockout after 5 failed attempts

4. Features Not to be Tested

- Real payment gateway / bank integration (payment is simulated within the program)
- Multi-user concurrent access and file locking (system is single-user, single-process)
- Networked multi-terminal / client-server operation (noted in SRS as a future extension only)
- Mobile applications
- Integration with actual IRCTC infrastructure
- GUI usability (no graphical front-end exists in this version — CLI only)

5. Test Approach / Strategy

Levels

- Unit tests (module-level: AuthModule, SearchModule, BookingModule, CancelModule, AdminModule, StorageModule, PaymentModule (sim.))
- Integration tests (e.g., BookingModule ↔ StorageModule, CancelModule ↔ BookingModule for waitlist promotion)
- System tests (end-to-end CLI flows: register → search → book → cancel → refund)
- Acceptance tests (against the SRS Section 6 acceptance test suites)

Types

- Functional testing (core features per Section 3)
- Regression testing (after each module change, particularly around PNR generation and waitlist promotion)
- Performance testing (search response time ≤ 1s for up to 5,000 train records, per RRS-NF-001)
- Reliability testing (invalid-input handling, abrupt-termination file consistency, per RRS-NF-002/003)
- Usability testing (plain-language CLI menus and error messages, per RRS-NF-004)

Entry Criteria: Stable build compiled with GCC/G++ (C/C++17), test data files (dummy trains, users, bookings) available, test environment ready.

Exit Criteria: 100% of planned test cases executed, 0 critical defects open, all High-priority functional requirements pass their reference test case, and the RTM in Section 13 shows every listed test case in a Passed state — consistent with the SRS Section 6 acceptance criteria.

5.1 Security Validation

- Verify passwords are stored hashed (not plaintext) in users.dat (RRS-SR-001)
- Verify password and dummy card-number input is masked on screen during entry (RRS-SR-002)
- Verify admin-only menu options are unreachable without a successful admin login (RRS-SR-003)
- Verify every admin action that changes train, fare or quota data is logged with a timestamp to admin_log.txt (RRS-SR-004)

- Verify a user account is locked for a cool-down period after 5 consecutive failed login attempts (RRS-SR-005)
- Fuzz malformed and out-of-range input fields (age, PNR, dummy card number) to confirm no crash or data loss

6. Test Environment

Hardware: Single-user desktop/laptop with standard keyboard and monitor/terminal; no specialised hardware required.

Software: Windows or Linux OS; GCC/G++ or equivalent C/C++17-capable compiler; C standard library / C++ STL; local disk storage for trains.dat, bookings.dat, users.dat, admin_log.txt.

Tools: GDB (debugging), Valgrind (memory/leak checks), a scripted input harness for fuzzing and load-timing runs, a hex/text viewer for password-storage checks, Git (version control), a defect tracker (e.g., a shared spreadsheet or GitHub Issues).

Test Data: Dummy passenger accounts and credentials, a seeded trains.dat with up to 5,000 records for performance testing, sample bookings across Confirmed/RAC/WL/Cancelled states, dummy 16-digit card numbers for the simulated payment step.

7. Test Schedule

Milestone	Date
Test case design	16-Sep-2026
Environment setup (compiler, test data seeding)	18-Sep-2026
Test execution start (unit & integration)	19-Sep-2026
System & performance testing	23-Sep-2026 to 26-Sep-2026
Security validation	27-Sep-2026
Acceptance testing (UAT / course evaluator review)	29-Sep-2026 to 30-Sep-2026

8. Test Deliverables

- Test Plan (this document)
- Test Cases (manual & scripted)
- Test Data (seeded trains.dat, users.dat, bookings.dat)
- Test Execution Logs
- Defect Reports
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Manapriya Kudurebyalya	Prepare plan, coordinate execution
Test Engineer	Pranav B N	Design & execute test cases, log defects
Developer	Amruta Karoshi	Support defect fixes and triage
Product Owner	Samaresh	Approve test results, sign-off readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build delivery ahead of a submission deadline	Request early smoke builds from the developer; freeze scope early
File corruption on abrupt termination (RRS-NF-003)	Run forced-kill tests early against a backup copy of test data files
PNR collisions after file reload (RRS-F-021 dependency)	Add dedicated regression tests around PNR generation after every reload
Limited team availability given academic schedule	Split test design/execution across team members by module ownership

11. Assumptions & Dependencies

- A compatible GCC/G++ (C/C++17) toolchain is available on the test machine
- Seeded test data (trains, users, bookings) will be prepared before execution begins
- The simulated Payment Module and simulated Payment Gateway behave deterministically for scripted tests
- No external database or network dependency exists, per the SRS constraints

12. Suspension & Resumption Criteria

Suspend testing if:

- Build fails to compile or crashes on startup
- Data files become corrupted and block more than 30% of planned test cases

Resume testing if:

- Blocking defects are resolved and a stable build is redelivered
- Test data files are restored to a known-good state

13. Test Case Management & Traceability

The RTM in the SRS (Section 8) maps each requirement to its implementing module and reference test case; this test plan tracks execution status against that same mapping. Representative examples:

Req ID	Requirement	Test Case(s)
RRS-F-001/002	Passenger registration & login	TC-Reg-01, TC-Auth-01
RRS-F-021	PNR generation (uniqueness)	TC-Book-02
RRS-F-023 / F-032	Waitlist/RAC queuing & auto-promotion	TC-Book-04, TC-Cancel-03
RRS-F-031	Refund slab calculation	TC-Cancel-02
RRS-NF-001	Search response time ≤ 1s	TC-Perf-01
RRS-SR-001/005	Password hashing & login lockout	TC-Sec-01, TC-Sec-05

14. Test Metrics & Reporting

Metrics collected

- % test cases executed
- % passed/failed
- Defect density (per module)
- Defect aging

- Requirement coverage against the SRS RTM

Reports

- Daily execution status (during the active test-execution window)
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead	Manapriya Kudurebyalya	Manapriya / 16-09-2026
Dev Lead	Amruta Karoshi	Amruta / 16-09-2026
Test Lead	Pranav B N	Pranav / 16-09-2026
Product Owner	Samaresh	Samaresh / 16-09-2026